CS2ON Coursework

Module Code: CS2ON

Assignment report Title: Linux Operating System and Building a small Office Network

Student Number (e.g. 25098635): 32025033

Actual hrs spent for the assignment: 12

Part A Task 1

Bash Script

```
#!/bin/bash
# Script: file_dir_management.sh

mkdir BOLU_DIR1 BOLU_DIR2 BOLU_DIR3

touch BOLU_DIR1/BOLU_001.txt BOLU_DIR1/BOLU_001_info.txt
touch BOLU_DIR2/BOLU_002.txt BOLU_DIR2/BOLU_002_info.txt
touch BOLU_DIR3/BOLU_003.txt BOLU_DIR3/BOLU_003_info.txt

echo "BOLU_DIR1 - Project Overview: This directory contains initial project ideas." >
BOLU_DIR1/BOLU_001.txt
echo "Details: This is related to the AI chatbot project." > BOLU_DIR1/BOLU_001_info.txt

echo "BOLU_DIR2 - Research: This directory is used for research documents." >
BOLU_DIR2/BOLU_002.txt
echo "Details: Contains findings from the user study." > BOLU_DIR2/BOLU_002_info.txt

echo "BOLU_DIR3 - Reports: This folder contains final reports." > BOLU_DIR3/BOLU_003.txt
echo "Details: Final version of the project report with all summaries." >
BOLU_DIR3/BOLU_003_info.txt

grep -r "project" BOLU_DIR*

ls -l BOLU_DIR1

cp BOLU_DIR1/BOLU_001.txt BOLU_DIR1/BOLU_001_copy.txt
mv BOLU_DIR1/BOLU_001_copy.txt BOLU_DIR1/BOLU_001_RENAMED.txt
```

Explanation of Linux Commands

Command	Purpose
mkdir	Creates directories.
touch	Creates new empty files.
echo "text" > file	Writes text to a file.
grep -r	Searches recursively for a keyword.
ls -l	Lists file details including permissions, size, owner.
cp, mv	Copy and rename files.
strace -e	Monitor system calls (open, write, execve).

The Answers for Part A Task 1 can be found in the appendix

Part A Task 2

2i. The pipe is created using the 'pipe(pfds)' system call at the start of the program and this call establishes a unidirectional communication channel between processes by creating two file descriptions stored in the 'pfds' array. Specifically, pfds[0]' is the read end of the pipe, and 'pfds[1]' is the write end and both descriptors allow data written by one process to be read by another. So, when traced using 'strace', the output typically shows something along the lines of 'pipe([3,4])', which indicates that file descriptor 3 is assigned to reading and 4 to writing. This setup forms the backbone of inter-process communication between the child and parent processes in the program.

2ii. A new process is created using 'fork()' system call, which duplicates the current process to create a child. In 'strace', this is shown as 'fork() = <pid>' which is the process ID of the newly created child. So therefore, the original process becomes the parent and execution continues simultaneously. Continually, with the 'if' process and log actions, both are processed separately using '[pid <number>]' prefixes, which helps to identify which process executes which part of the code.

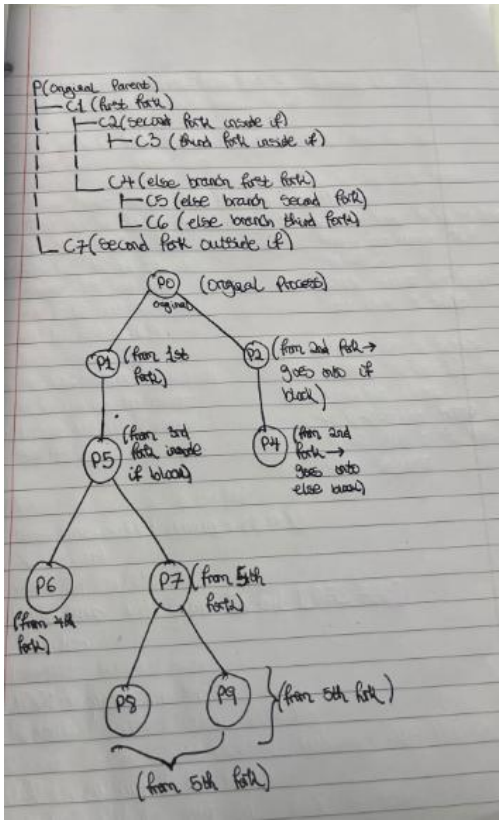
2iii. The 'cat file.txt' command is executed in the child process that is created by 'fork()' which is then shown in the child code path, where standard output is redirected to the pipe's write end and then 'execlp("cat", "cat", "file.txt", NULL)' is called. The parent process, however, executes 'execlp("grep", "grep", "pattern", NULL)' after redirecting its standard input to the pipe's read end. In the 'strace' output, the child process will show

‘execve("/bin/cat", ["cat", "file.txt"], ...)’ and the parent process will show ‘execve("/bin/grep", ["grep", "pattern"], ...)’. These commands therefore confirm which process is responsible for each action.

2iv. The ‘dup()’ and ‘close()’ system calls are used to redirect standard input and output for inter process communication and in the child process, ‘close(1)’ closes the standard output (file descriptor 1) and ‘dup(pfds[1])’ duplicates the pipe's write end to replace it, which means any output the child writes to ‘stdout’ will go into the pipe. Similarly, in the parent process, ‘close(0)’ closes standard input (file descriptor 0) and ‘dup(pfds[0])’ redirects it to read from the pipe. The ‘close()’ calls ensure unused ends of the pipe are closed, preventing interference or deadlocks and the system calls are essential for setting up a seamless data flow between processes.

Part A Task 3

3i. The program uses many ‘fork()’ system calls to create new processes, and each ‘fork()’ replicates the current process and so, the number of processes doubles, most of the time, with each ‘fork()’. So basically, the first ‘fork()’ creates 2 processes which are the original parent and child process and each of the 2 processes reaches the second ‘fork()’ inside the ‘if’ statement, if the result of ‘fork()’ is a non-zero value, they go into the ‘if’ block and do another ‘fork()’ or if the result is zero, they go into the ‘else’ block and do 2 more ‘fork()’ calls. So, the use of ‘fork()’ inside the conditional logic causes different processes to follow different paths, therefore creating a branching effect in the process tree.

3ii. 

3iii. The program creates many processes through a series of fork() system calls. At the beginning the program has a single parent process (P), which prints its own PID, then it executes the first fork(), creating one child process(C0), which then results in 2 processes: P and C0. Both processes continue with the if (fork()) statement, which acts as the second fork and at this stage, P forks again to create C1, and C0 forks to create C0a, therefore increasing the total number of processes to four. Continually, the third fork occurs within the if block. P, which is not the parent of C1, then forks once more to create C2, and C0(now the parent of C0a) then forks to create C0b, which results in 6 processes in total: P, C0, C1, C0a, C2, and C0b. Simultaneously, some processes enter the else block instead of the if block, depending in whether they are children or parent form the second fork. In the else block, both C1 and C0a execute 2 more fork() calls, the first fork in the else block causes C1 to create C1a, and C0a to create C0a1 which brings the total to 8 processes. The second fork in the else block further expands the process tree: C1 creates C1b, C1a creates C1a1, C0a creates C0a2, and C0a1 creates C0a1a, which adds 4 more processes, bringing the final total to 12 processes. The if (fork()) condition directs execution based on whether the process is a parent (non-zero return) or child (zero return), causing different branching behaviors. Processes that enter the if block create an additional child, while those in the else block create 4 due to 2 consecutive forks. As a result, each of the 12 processes reaches the final printf statement, printing its own PID and parent PID, however, the output order may vary depending on the scheduler, all processes will execute and print, producing 12 lines of output.

Part B Task 1

1. Figure 3.1 shows a small office LAN designed in Cisco Packet Tracer. At the centre of the network is a 2960-24TT switch that serves as the main hub for wired connection and it connects 6 office PCs (PC1 to PC6), a network printer, a server, and a wireless router. Each PC and the printer are connected via ethernet cables to the switch and the server, and they are also connected to the switch which functions as a RADIUS AAA server for wireless authentication. Continually, the WRT300N wireless router connects to the switch and serves as the default gateway and DHCP server for the entire network, as well as this, it provides wireless access to the tablet and the smartphone, both of which are connected using the WPA2 Enterprise security. This setup ensures that all devices, both wired and wireless, can communicate internally and access external networks through the router.
2. Figure 3.2 shows successful ping tests performed from wireless devices (Tablet PC1 and Smartphone1) to various wired devices on the same LAN, which shows that there is network connectivity and proper configuration. In the first image, Tablet PC1 sends ping requests to IP addresses 192.168.1.15 (PC6) and 192.168.1.11 (PC2). All four ICMP echo replies are successfully received for both addresses, with 0% packet

loss and average round trip times of 22ms and 18ms which confirms that the tablet can communicate effectively with the wired PCs. In the second image, Smartphone1 pings IP addresses 192.168.1.14 (PC5) and 192.168.1.12 (PC3). Again, all ping replies are received with 0% packet loss and average response times of 27ms and 22ms respectively. These results confirm that both wireless devices are correctly connected to the network via the wireless router using WPA2 Enterprise authentication and can communicate with other devices on the LAN.

Part B Task 2

The answers for Part B Task 2 can be found in the appendix along with the screenshots

Appendix

-Part A Task 1

```
mkdir -p BOLU_DIR1 BOLU_DIR2 BOLU_DIR3

touch BOLU_DIR1/BOLU_001.txt BOLU_DIR1/BOLU_001_info.txt
touch BOLU_DIR2/BOLU_002.txt BOLU_DIR2/BOLU_002_info.txt
touch BOLU_DIR3/BOLU_003.txt BOLU_DIR3/BOLU_003_info.txt

echo "BOLU_DIR1 - Project Overview: This directory contains initial project ideas." > BOLU_DIR1/BOLU_001.txt
echo "Details: This is related to the AI chatbot project." > BOLU_DIR1/BOLU_001_info.txt

echo "BOLU_DIR2 - Research:  This directory is used for research documents." > BOLU_DIR2/BOLU_002.txt
echo "Details: Contains findings from the user study." > BOLU_DIR2/BOLU_002_info.txt

echo "BOLU_DIR3 - Reports: This folder contains final reports." > BOLU_DIR3/BOLU_003.txt
echo "Details: Final version of the project report with all summaries." > BOLU_DIR3/BOLU_003_info.txt

grep -r "project" BOLU_DIR*


ls -l BOLU_DIR1


cp BOLU_DIR1/BOLU_001.txt BOLU_DIR1/BOLU_001_copy.txt
mv BOLU_DIR1/BOLU_001_copy.txt BOLU_DIR1/BOLU_001_RENAMED.txt
```

Figure 1.1 -This is the bash script 'file_dir_management.sh' which automates the creation and management of directories and files using the Linux commands.

```
BOLU_DIR1/BOLU_001_RENAMED.txt:BOLU_DIR1 - Project Overview: This directory contains initial project ideas.
BOLU_DIR1/BOLU_001_info.txt:Details: This is related to the AI chatbot project.
BOLU_DIR1/BOLU_001.txt:BOLU_DIR1 - Project Overview: This directory contains initial project ideas.
BOLU_DIR3/BOLU_003_info.txt:Details: Final version of the project report with all summaries.
total 7
-rw-r--r-- 1 vy025033 siu 52 May  9 14:29 BOLU_001_info.txt
-rw-r--r-- 1 vy025033 siu 77 May  9 14:26 BOLU_001_RENAMED.txt
-rw-r--r-- 1 vy025033 siu 77 May  9 14:29 BOLU_001.txt
```

Figure 1.2 -This is the output when './file_dir_management.sh' is run. I used 'nano' to edit the file and './' to run the file

Figure 1.3- This shows the 2 files created inside each of the directories and they have been generated using my part of my name 'BOLU', a three-digit sequence of 001,002 and 003 and the purpose is to have one main content file and one supplementary file in each directory.

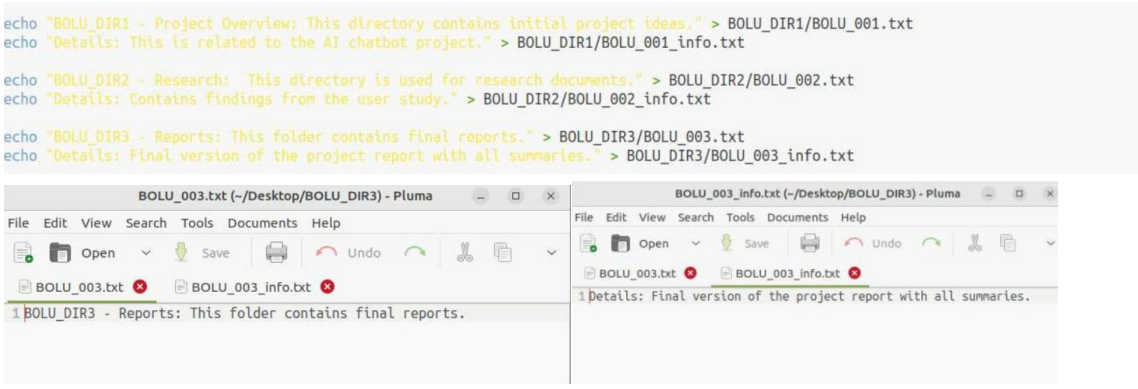


Figure 1.4- This shows the code used to create the 'BOLU_003.txt' file and the contents of the file

Figure 1.5- In these screenshots, 'grep' is a command that searches for patterns and keyword inside of files and the '-r' makes the search recursive, so it looks inside all the sub directories and files under 'BOLU_DIR*' and the keyword being searched for is 'project' which was included as part of the sample content added to some .txt files. So, using 'BOLU_DIR*', the command is told to search in all directories starting with 'BOLU_DIR'. The output of 'grep -r "project" BOLU_DIR* is shown in figure 1.5 and it shows that the content was correctly added to the files and that grep can search and identify matching patterns across files.

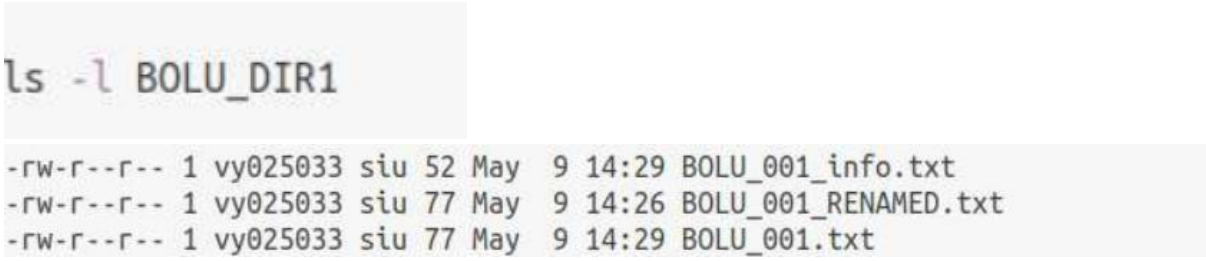


Figure 1.6- In this screenshot, to list the contents of one directory with detailed information, I used the command 'ls -l BOLU_DIR1', which displayed each of the file's permissions, the

number of hard links, owner and group, file size in bytes, last modified date and time, and file name. For example, the file ‘BOLU_001.txt’ and ‘BOLU_001_info.txt’ were listed with their respective details including ‘rw-r--r—' permissions, ownership by the user, and sizes of 52 and 77 bytes.

```
cp BOLU_DIR/BOLU_001.txt BOLU_DIR1/BOLU_001_copy.txt
mv BOLU_DIR1/BOLU_001_copy.txt BOLU_DIR1/BOLU_001_RENAMED.txt
```

```
-rw-r--r-- 1 vy025033 siu 52 May  9 14:35 BOLU_001_info.txt
-rw-r--r-- 1 vy025033 siu 77 May  9 14:29 BOLU_001_RENAMED.txt
-rw-r--r-- 1 vy025033 siu 77 May  9 14:35 BOLU_001.txt
```

Figure 1.7- This shows how the cp and mv commands were used to copy content from one file to another and rename by copying ‘BOLU_001.txt’ and renaming the copy. The file ‘BOLU_001_RENAMED.txt’ was created with identical content and size (77 bytes), confirming the copy was successful which was achieved by using cp BOLU_DIR1/BOLU_001.txt BOLU_DIR1/BOLU_001_copy.txt followed by mv BOLU_DIR1/BOLU_001_copy.txt BOLU_DIR1/BOLU_001_RENAMED.txt. Both files appear in the directory listing of BOLU_DIR1, which shows that the content was copied, and file renamed.

```
openat(AT_FDCWD, "BOLU_DIR1/BOLU_001.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
openat(AT_FDCWD, "BOLU_DIR1/BOLU_001_info.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
openat(AT_FDCWD, "BOLU_DIR2/BOLU_002.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
openat(AT_FDCWD, "BOLU_DIR2/BOLU_002_info.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
openat(AT_FDCWD, "BOLU_DIR3/BOLU_003.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
openat(AT_FDCWD, "BOLU_DIR3/BOLU_003_info.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
```

```
execve("./file_dir_management.sh", ["./file_dir_management.sh"], 0x7ffeb442c4e8 /* 61 vars */) = 0
strace: Process 3704868 attached
[pid 3704868] execve("/usr/bin/mkdir", ["mkdir", "-p", "BOLU_DIR1", "BOLU_DIR2", "BOLU_DIR3"], 0x55db8c8c15c0 /* 61 vars */) = 0
[pid 3704868] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=3704868, si_uid=146293, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 3704870 attached
[pid 3704870] execve("/usr/bin/touch", ["touch", "BOLU_DIR1/BOLU_001.txt", "BOLU_DIR1/BOLU_001_info.txt"], 0x55db8c8dafa0 /* 61 vars */) = 0
[pid 3704870] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=3704870, si_uid=146293, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 3704872 attached
[pid 3704872] execve("/usr/bin/touch", ["touch", "BOLU_DIR2/BOLU_002.txt", "BOLU_DIR2/BOLU_002_info.txt"], 0x55db8c8dafa0 /* 61 vars */) = 0
[pid 3704872] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=3704872, si_uid=146293, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 3704873 attached
[pid 3704873] execve("/usr/bin/touch", ["touch", "BOLU_DIR3/BOLU_003.txt", "BOLU_DIR3/BOLU_003_info.txt"], 0x55db8c8dafa0 /* 61 vars */) = 0
[pid 3704873] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=3704873, si_uid=146293, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 3704878 attached
[pid 3704878] execve("/usr/bin/grep", ["grep", "-r", "project", "BOLU_DIR1", "BOLU_DIR2", "BOLU_DIR3"], 0x55db8c8dafa0 /* 61 vars */) = 0
```

```
write(1, "BOLU_DIR1 - Project Overview: Th"..., 77) = 77
write(1, "Details: This is related to the "..., 52) = 52
write(1, "BOLU_DIR2 - Research:  This dire"..., 70) = 70
write(1, "Details: Contains findings from "..., 48) = 48
write(1, "BOLU_DIR3 - Reports: This folder"..., 57) = 57
write(1, "Details: Final version of the pr"..., 65) = 65
```

Figure 1.8- In these screenshots, to analyse system-level operations of the Bash script, strace was run with three filters: openat, write, and execve. The openat filter showed system calls made to open or create files which is confirmed when the script used commands like touch and echo. The write filter displayed how content was written into files, providing visibility into file manipulation actions and finally, the execve filter traced external command executions such as grep, ls, and mv, confirming how the script interacts with the underlying

operating system. These traces verify that the script performs the expected system-level operations.

Part A Task 2

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main(void) {
   int pfds[2];
   pipe(pfds); //Create a pipe
   if(!fork()){ //child process
      close(1); //Close stdout
      dup(pfds[1]); //Redirect stdout to pipe write end
      close(pfds[0]); //Close read end of the pipe
      execlp("cat","cat","file.txt",NULL); //Execute 'cat file.txt
}else { //Parent process
  close(0); //Close stdin
  dup(pfds[0]); //Redirect stdin to pipe read end
  close(pfds[1]); //Close write end of the pipe
  execlp("grep","grep","pattern",NULL); //Execute 'grep pattern'
}
 return 0;
}
```

Figure 2.1- This is the bash script file names ‘pipe_exec.c’ which demonstrates inter-process communication using a pipe.

```
vy025033@nxnode3:~/Desktop$ gcc pipe_exec.c -o pipe_exec
vy025033@nxnode3:~/Desktop$ echo "this is a pattern line" > file.txt
vy025033@nxnode3:~/Desktop$ echo "another line " >> file.txt
vy025033@nxnode3:~/Desktop$ cat file.txt
this is a pattern line
another line
```

Figure 2.2- This screenshot shows the compilation of the ‘pipe_exec.c’ program and the creation of a test input file named file.txt. The command ‘gcc pipe_exec.c -o pipe_exec’ compiles the C program into an executable file names ‘pipe_exec’. Continually, 2 echo commands are used to write sample lines to file.txt, including a line containing the keyword "pattern" which is later searched by the program and finally the ‘cat file.txt’ command confirms the contents of the file, ensuring the test data is correctly set up for the pipe-based inter-process communication.

```
vy025033@nxnode3:~/Desktop$ strace -f -e  trace=fork,execve,pipe,dup,close ./pipe_exec
execve("./pipe_exec", ["./pipe_exec"], 0x7ffd69eddd88 /* 61 vars */) = 0
close(3)                                = 0
close(3)                                = 0
strace: Process 2595605 attached
[pid 2595604] close(0)                  = 0
[pid 2595604] dup(3)                    = 0
[pid 2595604] close(4 <unfinished ...>
[pid 2595605] close(1 <unfinished ...>
[pid 2595604] <... close resumed>)      = 0
[pid 2595605] <... close resumed>)      = 0
[pid 2595604] execve("/usr/local/sbin/grep", ["grep", "pattern"], 0x7fff627f6488 /* 61 vars */ <unfinished ...>
[pid 2595605] dup(4)                    = 1
[pid 2595604] <... execve resumed>)     = -1 ENOENT (No such file or directory)
[pid 2595605] close(3 <unfinished ...>
[pid 2595604] execve("/usr/local/bin/grep", ["grep", "pattern"], 0x7fff627f6488 /* 61 vars */ <unfinished ...>
[pid 2595605] <... close resumed>)      = 0
[pid 2595604] <... execve resumed>)     = -1 ENOENT (No such file or directory)
[pid 2595605] execve("/usr/local/sbin/cat", ["cat", "file.txt"], 0x7fff627f6488 /* 61 vars */ <unfinished ...>
[pid 2595604] execve("/usr/sbin/grep", ["grep", "pattern"], 0x7fff627f6488 /* 61 vars */) = -1 ENOENT (No such file or directory)
[pid 2595605] <... execve resumed>)     = -1 ENOENT (No such file or directory)
[pid 2595604] execve("/usr/bin/grep", ["grep", "pattern"], 0x7fff627f6488 /* 61 vars */ <unfinished ...>
[pid 2595605] execve("/usr/local/bin/cat", ["cat", "file.txt"], 0x7fff627f6488 /* 61 vars */) = -1 ENOENT (No such file or directory)
[pid 2595605] execve("/usr/sbin/cat", ["cat", "file.txt"], 0x7fff627f6488 /* 61 vars */) = -1 ENOENT (No such file or directory)
[pid 2595605] execve("/usr/bin/cat", ["cat", "file.txt"], 0x7fff627f6488 /* 61 vars */ <unfinished ...>
[pid 2595604] <... execve resumed>)     = 0
[pid 2595604] close(4)                  = 0
[pid 2595604] close(4)                  = 0
[pid 2595605] <... execve resumed>)     = 0
```

```
[pid 2595605] <... execve resumed>)     = 0
[pid 2595604] close(4)                  = 0
[pid 2595605] close(3)                  = 0
[pid 2595604] close(4)                  = 0
[pid 2595605] close(3)                  = 0
[pid 2595604] close(4)                  = 0
[pid 2595604] close(4)                  = 0
[pid 2595605] close(3)                  = 0
[pid 2595604] close(4)                  = 0
this is a pattern line
[pid 2595605] close(3)                  = 0
[pid 2595605] close(1)                  = 0
[pid 2595605] close(2)                  = 0
[pid 2595605] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=2595605, si_uid=146293, si_status=0, si_utime=0, si_stime=0} ---
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

Figure 2.3- This captures the system call trace for the execution of the ‘pipe_exec’ code using strace with the ‘-f’ flag to follow the child processes and filters for fork, execution, execve, pipe, dup, and close. It shows the key actions that occur during the inter-process communication setup. The ‘fork()’ call creates a child process that is verified by strace attaching to a new PID and the ‘pipe()’ system call sets up a communication channel with file descriptors used by both processes. Both the ‘dup()’ and ‘close()’ functions called in the parent and child processes demonstrate redirection of standard input and output through the pipe. Finally, ‘execve()’ shows the child executing the ‘cat file.txt’ command and the parent executing the grep pattern which confirms that the process behaviour is correct which validates that the C program correctly sets up a pipe and redirects I/O between two processes.

Part B Task 1

Figure 3.1

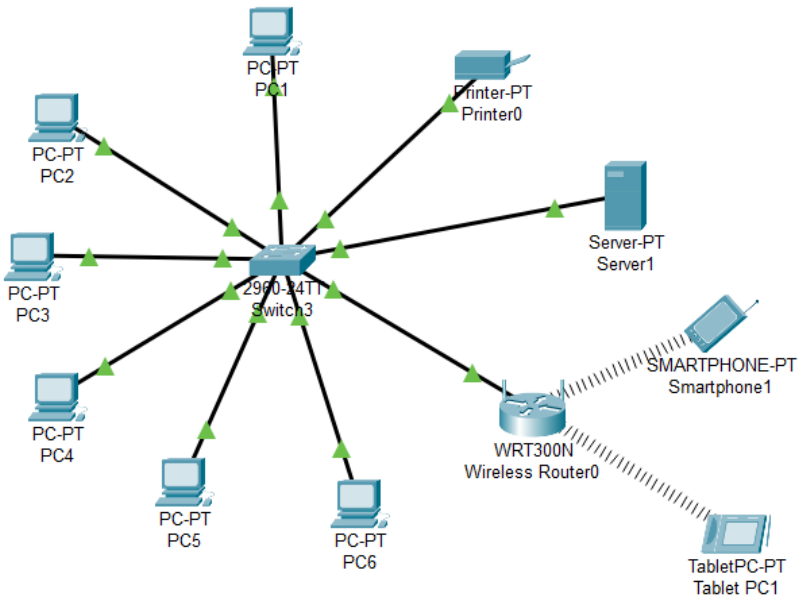
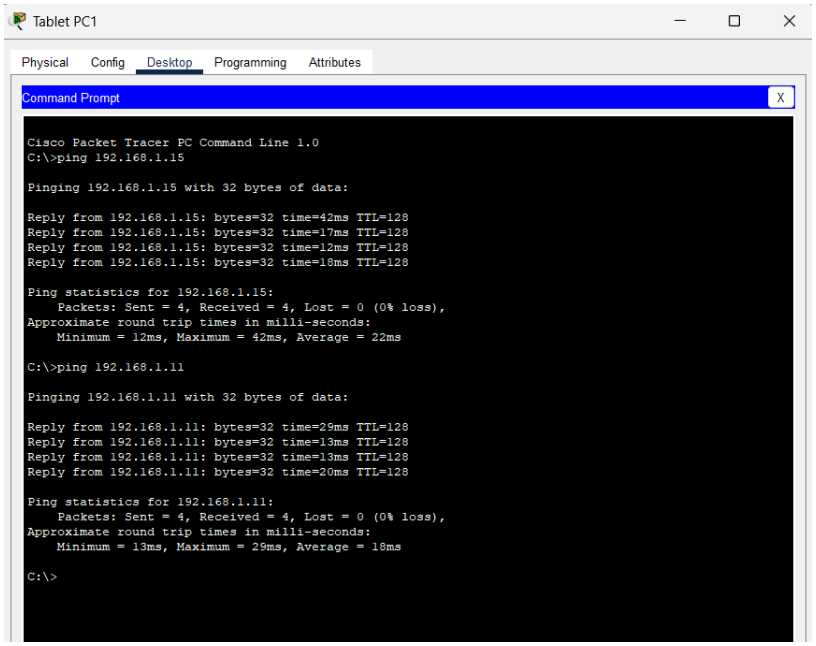
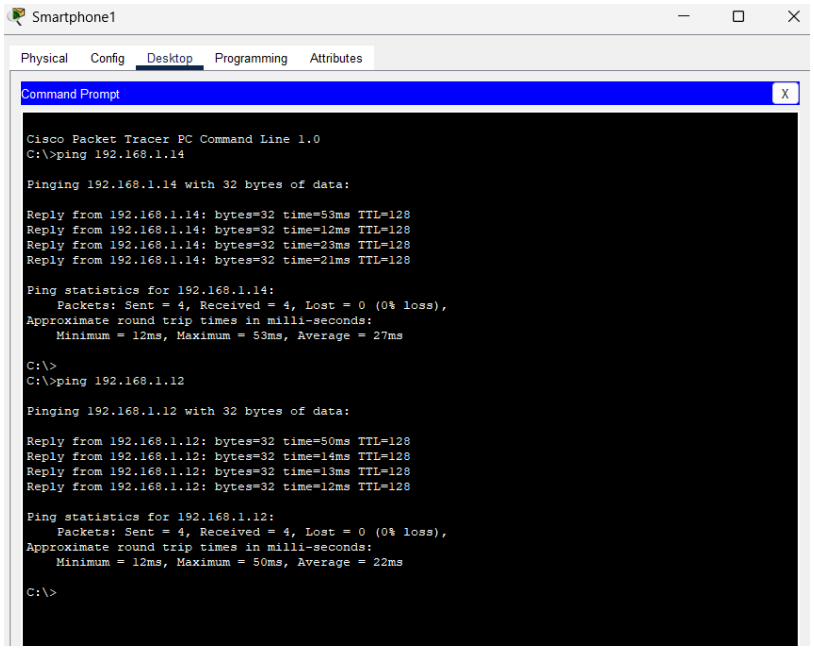
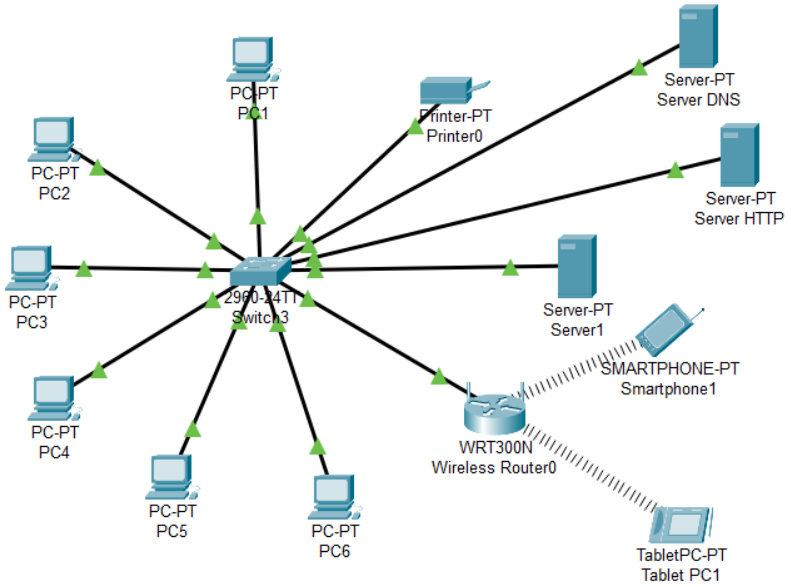

PC-PT
PC1
Printer-PT
Printer0
PC-PT
PC2
Server-PT
Server1
PC-PT
PC3
2960-24TT
Switch3
SMARTPHONE-PT
Smartphone1
PC-PT
PC4
WRT300N
Wireless Router0
PC-PT
PC5
PC-PT
PC6
TabletPC-PT
Tablet PC1

Figure 3.2





Part B Task 2



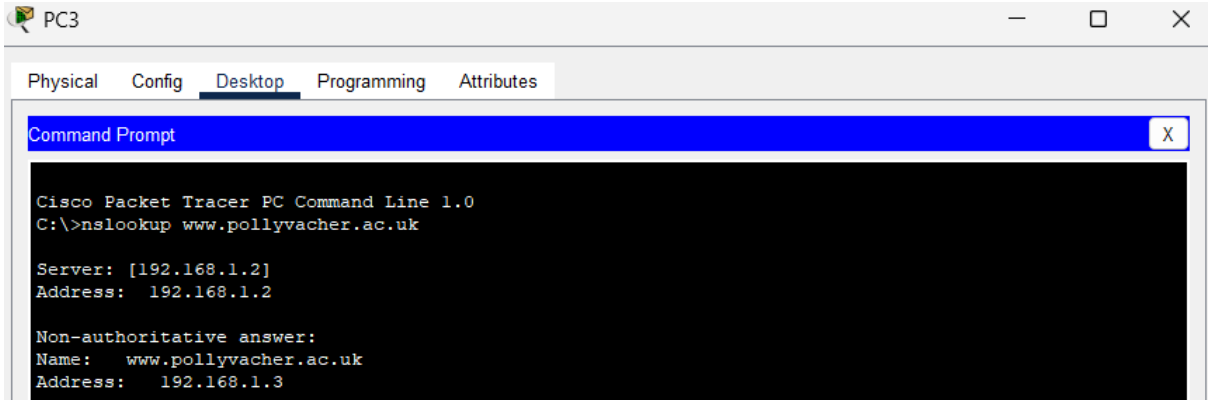


Figure 3.3- In this figure, the DNS resolution test was done successfully on PC3 using the 'nslookup' command. The command 'nslookup www.pollyvacher.ac.uk' was entered in the command prompt and the output showed that the DNS server at IP address 192.168.1.2 correctly resolved the domain name to the HTTP server's IP address 192.168.1.3, which verifies the fact that the DNS server was properly configured with an A record mapping the domain name to the HTTP server, as well as this, It also confirms that the PC was using the correct DNS server and that DNS communication within the network is functioning as it is supposed to.

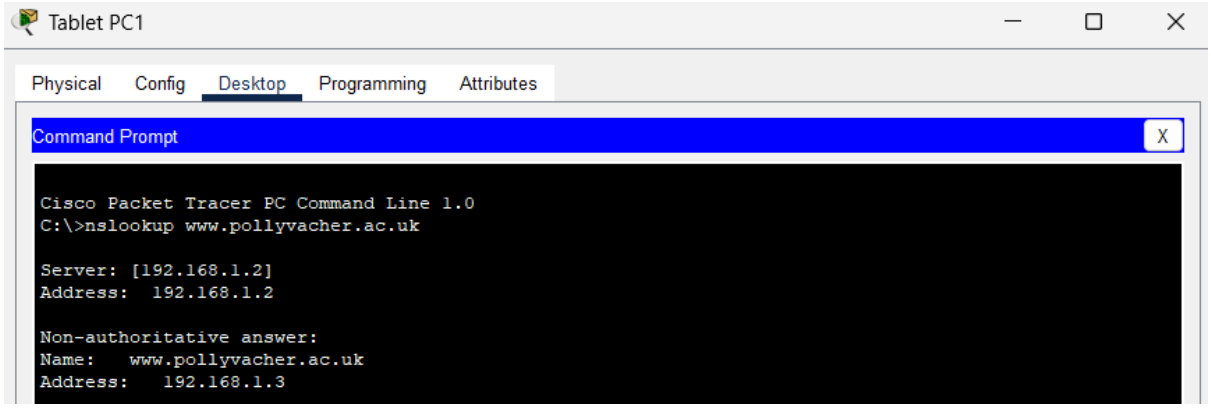


Figure 3.4-In this figure, the DNS resolution was also successfully tested on the tablet, using the 'nslookup' command. When the command 'nslookup www.pollyvacher.ac.uk' was entered in the tablet's command prompt, the output showed that the DNS server at 192.168.1.2 responded with the correct IP address 192.168.1.3 for the domain 'www.pollyvacher.ac.uk', which confirms that the tablet was properly connected to the wireless network, received the correct DNS server configuration and was also able to resolve domain names to IP addresses. This shows that DNS resolution is working not only for wired clients but also for wireless devices on the network.

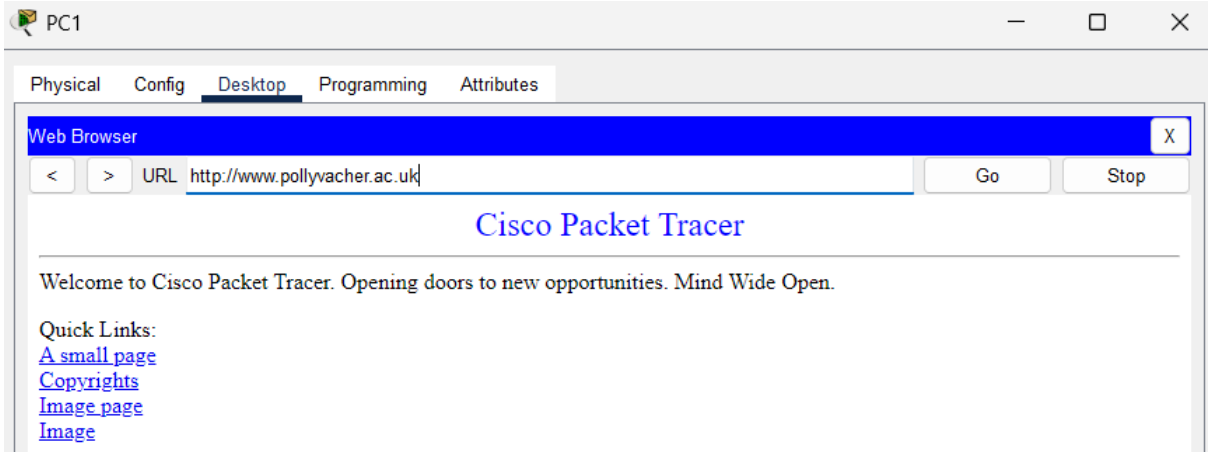


Figure 3.5-In this figure, to verify HTTP functionality, a test was conducted on one of the PCs using the built-in web browser. The URL 'www.pollyvacher.ac.uk' as entered the browser's address bar and the browser successfully loaded the default web page hosted on the HTTP server at 192.168.1.3, which confirms that the A record configured on the DNS server correctly directed the domain name to the web server's IP address. This result shows that the DNS and HTTP services were both properly configured, and that the PC could successfully access the website over the network using domain name resolution instead of a direct IP address.

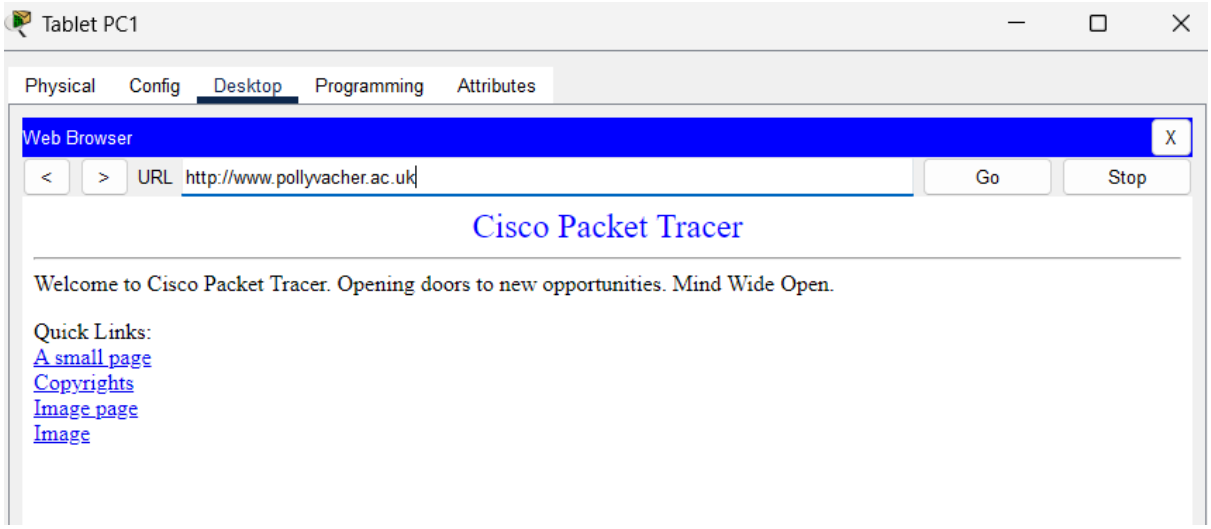


Figure 3.6- In this figure, the final test involved verifying HTTP access from the tablet which is a wireless mobile device connected to the network. Using the tablet's web browser, the URL 'www.pollyvacher.ac.uk' was entered into the address bar and the page successfully loaded, displaying the default content served by the HTTP server at IP address 192.168.1.3. This confirmed that the tablet was able to resolve the domain name via the DNS server and access the web server over the wireless connection. The success of this test demonstrates that both DNS and HTTP services were fully functional for wireless clients, completing the network validation process.